

Programming Cellular Automata in C and MLX

Patrik Christen^{1,2}

¹ FHNW, Institute for Information Systems, Olten, Switzerland

² ETH Zurich, Chair for Philosophy, Zurich, Switzerland

`patrik.christen@fhnw.ch`

Abstract. Cellular automata are capable of simulating a wide range of phenomena and at the same time are well suitable for parallel computing, even in an energy-efficient manner. They are thus promising models for edge computing and high-performance desktop computing. While user-friendly machine learning Python frameworks allow programmers to implement performant machine learning code, often optimised for specific hardware, such an easy-to-use and optimised hardware and software combination is still lacking for the programming of cellular automata. The recently released MLX array framework to program Apple silicon hardware seems promising to improve this situation, especially because it provides user-friendly operations for parallel processing on the GPU and optimised compilation. The present study therefore explores MLX for the programming of cellular automata and compares running times for different grid sizes on the CPU and GPU with a basic C implementation on the CPU. It is found that MLX versions for the CPU and GPU show similar running times and that compilation leads to minor improvements in both cases. The C version is up to 2200-times faster as the compiled MLX version and can handle one order of magnitude larger grid sizes. The study concludes that the MLX framework in its current stage is unsuitable to program performant cellular automata for edge computing and high-performance desktop computing, however, that easy-to-program optimisations look promising in MLX and C. This is because the Apple silicon architecture with its multicore CPU and GPU has much to offer for parallel programming.

Keywords: Elementary Cellular Automata · C · MLX · Edge Computing · High-Performance Desktop Computing.

1 Introduction

It is known since a long time that cellular automata can simulate a wide range of behaviours including complex behaviour mimicking intricate patterns as observed in nature [20]. More recently, it has been even demonstrated that cellular automata are not only abstract models of computation but are also implemented by nature, i.e. as found in the development of colour patterns in lizard skin [13]. It is therefore not surprising that cellular automata are capable of simulating phenomena from a wide range of applications such as lava flow [19], wildfire

propagation [9], random number generation [14, 21], image processing [6, 17], and image-based simulation [12]. Also the simulation of physical phenomena in general such as phase transition, fluid flow, and reaction-diffusion processes has been shown to be well suited to be simulated with cellular automata [7, 11, 18, 22].

Besides the usefulness for simulating many different phenomena, cellular automata are also known for their parallel computing characteristic. They are well suited to be implemented on hardware capable of parallel processing, which has been done, e.g. for general purpose graphics processing units (GPGPUs) [10, 15] and field-programmable gate arrays (FPGAs) [5, 16]. The latter can be designed for cellular automata in such a way to provide a more energy-efficient alternative to artificial intelligence methods such as deep learning [16]. Cellular automata running on such hardware seems to be a promising computing approach for edge computing [16] and also high-performance desktop computing [8].

These exciting hardware developments allow exploiting cellular automata characteristics like parallelism, however, they usually make the programming more demanding. Widespread and easy-to-use frameworks such as currently used in machine learning are still not available. Google’s TensorFlow [1] is a prominent example where it is possible to program in Python notebooks in the browser and execute the program on so-called tensor processing units (TPUs) that implement matrix multiplication in hardware. The transformation of Python code into performant and often optimised code for the hardware is done automatically. Apple recently launched their own machine learning framework, called MLX [2], to allow user-friendly programming of Apple silicon hardware. It comes with a Python API, which includes a number of predefined operations such as matrix multiplication. It also offers some interesting features for parallel programming of the Apple silicon GPU with unified memory [4] as well as a compilation operation that allows optimised compilation of the Python code [3]. If MLX provides user-friendly programming of operations for cellular automaton simulations, Apple would potentially provide a performant hardware and software combination for the programming of cellular automata, especially for edge computing and high-performance desktop computing.

In the present study, it is therefore explored whether MLX and Apple silicon hardware are suitable for the programming of cellular automata. In particular, it is tested whether unified memory and compilation on the central processing unit (CPU) and GPU can be exploited to improve running times in comparison to a basic C implementation on the CPU. In section 2, the study starts with a short description of the MLX array framework followed by a description of the used elementary cellular automata in section 3. Section 4 and 5 show how cellular automata can be programmed in MLX and C, respectively. Section 6 describes the simulation experiments and their results. Finally, results, limitations, and future work are discussed in section 7.

2 MLX Array Framework

Apple recently released a machine learning framework referred to as MLX. It is an array framework specifically developed for machine learning on Apple silicon and it is intended to be user-friendly and at the same time efficient [2]. It provides a number of features such as a user-friendly Python API, function transforms, lazy evaluation, compilation, and unified memory [2].

Unified memory and compilation are relevant here and thus explained in some more detail. Unified memory means that the CPU and GPU have access to the same memory pool [4]. The following example Python code shows how the arrays `a` and `b` can be defined in unified memory and how they can be used in operations such as `add()` to be executed on the CPU or GPU device.

```
import mlx.core as mx
a = mx.array([1, 2, 3, 4])
b = mx.array([5, 6, 7, 8])
mx.add(a, b, stream=mx.cpu)
mx.add(a, b, stream=mx.gpu)
```

It is also possible to define a default device with `mx.set_default_device(mx.gpu)`. The compilation feature allows compiling a function `fun()` with `fun_compiled = mx.compile(fun)`, which merges common work and fuses certain operations [3].

3 Elementary Cellular Automata

In the present study, elementary cellular automata are used. They discretise space in a one-dimensional grid in which each cell gets updated synchronously according to the same predefined update rule in discrete time steps. States are either on or off and therefore have a boolean type. The update rule defines the state of a cell in the next step according to the cell's neighbouring cell states and the cell's current state. Elementary cellular automata have been thoroughly investigated and described before, e.g. by Stephen Wolfram [22] and Andrew Ilachinski [11].

4 MLX Cellular Automaton

The MLX and the C version were programmed in the same way without any optimisation to make them comparable, at least with the compiled MLX version. Two boolean MLX arrays, one for the initial grid and one for the updated grid, are first created.

```
grid_size = 1001
grid_0 = mx.zeros((grid_size), mx.bool_)
grid_1 = mx.zeros((grid_size), mx.bool_)
```

Next, an update function is created for updating grid cells according to a predefined rule. Here, rule 250 is used because it can be expressed by simple arithmetic, i.e. $l + r - l * r$, where l is the state of the left neighbour, r the state of the right neighbour, and c the state of the current cell [22]. MLX operations were used to program the rule and a compiled version was created as well.

```
def update_rule250(left , center , right):
    return mx.subtract(mx.add(left , right , stream=mx.gpu) ,
        mx.multiply(left , right , stream=mx.gpu) ,
        stream=mx.gpu)

update_rule250_compiled = mx.compile(update_rule250)
```

Finally, to perform an update iteration, the update function above was called in a for loop running over the MLX arrays.

```
for i in range(1, grid_size-1, 1):
    grid_1[i] = update_rule250(grid_0[i-1],
        grid_0[i], grid_0[i+1])
```

5 C Cellular Automaton

As already mentioned above, MLX and C cellular automata were programmed in the same basic way. Also in the C version, two boolean arrays are first created.

```
int grid_size = 1001;
bool grid_0[grid_size];
bool grid_1[grid_size];
```

In the next step, again rule 250 is programmed in an update function since it can be expressed by simple arithmetic.

```
bool update_rule250(bool l, bool c, bool r)
{
    return l + r - l * r;
}
```

And finally, again a for loop is used to update the state of each cell of the grid in one iteration.

```
for(int i=1; i<grid_size-1; ++i)
{
    grid_1[i] = update_rule250(grid_0[i-1],
        grid_0[i], grid_0[i+1]);
}
```

6 Simulation Experiments

Elementary cellular automaton experiments simulating rule 250 for one iteration were run on a Apple MacBook Air M2 16 GB with different MLX configurations and C compilers as well as different cellular automaton grid sizes. The MLX Python API was used with MLX 0.10.0 and Python 3.12.3. MLX experiments included a CPU and a GPU version as well as their compiled versions. C experiments were compiled with Apple clang 15.0.0 and GNU gcc 13.3.0. For each of these six cases, cellular automaton grid sizes of 10^3 , 10^4 , and 10^5 elements were run and the running time measured. For the C versions also grid sizes of 10^6 and $4.1 * 10^6$ were simulated. Higher grid sizes resulted in segmentation faults in the C version and MLX versions had to be canceled after several hours of running time without showing any results.

Simulation results are given in figure 1 below. Please note that the left and right bar charts are given with different axis values because of the large differences between measured running times. Comparing running times of the MLX versions reveals similar times for the CPU and GPU versions, and a slight improvement (approx. factor 1.1) of the compiled over the uncompiled versions for the CPU and GPU. Comparing running times of the C versions reveals faster times for the GNU gcc over the Apple clang compiler (approx. factor 1.5). Finally comparing the running times between the C and MLX versions, specifically the MLX CPU compiled versus the GNU gcc version, reveals faster times for the C version (approx. factor 1250). The running time factor between MLX and C increases with increasing grid size from approx. factor 1250 to 1500 and to 2200 for grid sizes of 10^3 , 10^4 , and 10^5 , respectively.

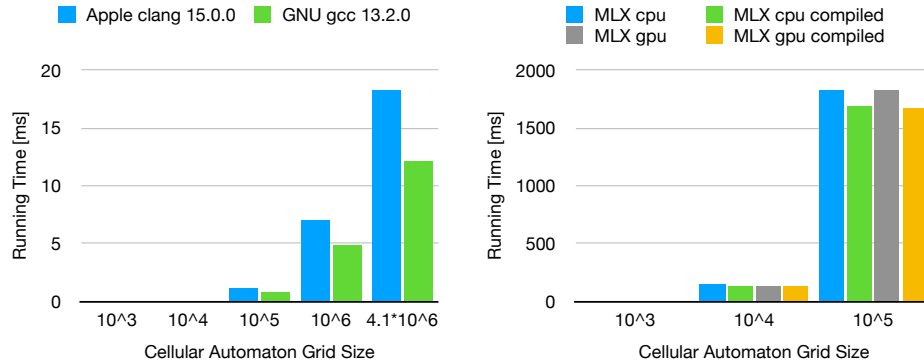


Fig. 1. Running times in milliseconds of C and MLX cellular automata simulating rule 250 for one iteration.

7 Discussion

Many applications could profit from a cellular automaton approach since it is capable of simulating a wide range of phenomena and has energy-efficient and parallel computing characteristics. Edge computing and high-performance desktop computing are particularly interesting applications, however, programming cellular automata to exploit these characteristics is challenging and user-friendly but still performant frameworks as currently used in machine learning are missing.

Here, Apple’s new MLX array framework with unified memory for CPU and GPU, and compilation features is explored for the programming of cellular automata and compared to a basic C implementation on the CPU. It is found that MLX CPU and GPU versions show similar running times for different grid sizes and the compilation slightly improves both versions as they are 1.1-times faster. In contrast, the choice of C compiler seems to have a bigger effect, with the GNU gcc compiler 1.5-times faster as the Apple clang compiler. Finally, the GNU gcc version is up to 2200-times faster for larger grid sizes as compared to the compiled MLX version. Regarding memory, MLX seems to require more memory as it was not possible to run larger grid sizes than 10^5 , whereas the C version could run up to a grid size of $4.1 \cdot 10^6$.

Comparing the running times shows that the C versions are drastically faster despite their basic implementation for the CPU. Not only that, since it only requires basic knowledge of C, it is as easy to program as with the MLX Python API. MLX is still in an early stage, however, the current framework seems unsuitable for the programming of cellular automata. It might be possible to create faster MLX versions by using certain highly optimised machine learning operations from the framework to program cellular automata in some way. This was, however, not the intention in this first study because the aim would be to have something user-friendly. It would certainly be interesting to experiment with easy-to-program optimisations in the C and MLX versions, especially to make more use of Apple’s multicore CPU and GPU architecture. Most performant would arguably be the direct programming of Metal performance shaders for the GPU, however, this is likely not that easy to program.

Also regarding memory usage, the MLX versions are behind the C versions, although the difference is not as drastic as for running times. The issue here is that so far computers with the Apple silicon architecture have very limited memory capacities. The M2 Ultra of the Mac Pro, for example, can be configured with up to 192 GB, which is much less as the 1.5 TB limit of the Intel-based Mac Pro. It is thus of great importance to carefully deal with memory in a framework like MLX. It seems, however, that this is not the case yet, at least not for the programming of cellular automata.

Besides the comparison between MLX and C versions, it is interesting to notice the difference in running times between clang and gcc compilers. One would expect clang to be best optimised for the machine as it comes from Apple. It might be possible to gain similar running times if the compilation is optimised, but again this would not be in the spirit of this study.

This study concludes that Apple’s new MLX array framework with a Python API is in its current stage unsuitable to program performant cellular automata for edge computing and high-performance desktop computing. The Apple silicon architecture with multicore CPU and GPU is, however, still interesting to exploit with easy-to-program optimisations in MLX and C for these applications.

Acknowledgments. The Santa Fe Institute is gratefully acknowledged for hosting the author in Autumn 2023.

Disclosure of Interests. The author has no competing interests to declare that are relevant to the content of this article.

References

1. An Open Source Machine Learning Framework for Everyone, <https://github.com/tensorflow/tensorflow> [Accessed: 19 April 2024]
2. MLX: An array framework for Apple silicon, <https://github.com/ml-explore/mlx> [Accessed: 20 April 2024]
3. MLX: Compilation, <https://ml-explore.github.io/mlx/build/html/usage/compile.html> [Accessed: 21 April 2024]
4. MLX: Unified Memory, https://ml-explore.github.io/mlx/build/html/usage/unified_memory.html [Accessed: 21 April 2024]
5. Bakhteri, R., Cheng, J., Semmelhack, A.: Design and Implementation of Cellular Automata on FPGA for Hardware Acceleration. *Procedia Computer Science* **171**, 1999–2007 (2020)
6. Cappellari, L., Milani, S., Cruz-Reyes, C., Calvagno, G.: Resolution Scalable Image Coding With Reversible Cellular Automata. *IEEE Transactions on Image Processing* **20**(5), 1461–1468 (2011)
7. Chopard, B.: Cellular Automata Modeling of Physical Systems. In: Meyers, R.A. (ed.) *Computational Complexity: Theory, Techniques, and Applications*. pp. 407–433. Springer, New York (2012)
8. Christen, P.: Programming Data Structures for Large-Scale Desktop Simulations of Complex Systems. In: 2022 IEEE International Conference on Systems, Man, and Cybernetics (SMC). pp. 3000–3005 (2022)
9. Freire, J.G., DaCamara, C.C.: Using cellular automata to simulate wildfire propagation and to assist in fire management. *Natural Hazards and Earth System Sciences* **19**(1), 169–179 (2019)
10. Gobron, S., Bonafos, H., Mestre, D.: GPU Accelerated Computation and Visualization of Hexagonal Cellular Automata. In: Umeo, H., Morishita, S., Nishinari, K., Komatsuzaki, T., Bandini, S. (eds.) *Cellular Automata*. pp. 512–521. Springer, Berlin, Heidelberg (2008)
11. Ilachinski, A.: *Cellular Automata: A Discrete Universe*. World Scientific, New Jersey, London, Singapore, Hong Kong (2001)
12. Keller, F., Christen, P.: Bone as a complex system: Computing bone remodelling across biological hierarchies. In: 8th World Congress of Biomechanics (WCB) (2018)
13. Manukyan, L., Montandon, S.A., Fofonjka, A., Smirnov, S., Milinkovitch, M.C.: A living mesoscopic cellular automaton made of skin scales. *Nature* **544**, 173–179 (2017)

14. Manzoni, L., Mariot, L.: Cellular Automata Pseudo-Random Number Generators and Their Resistance to Asynchrony. In: Mauri, G., El Yacoubi, S., Dennunzio, A., Nishinari, K., Manzoni, L. (eds.) *Cellular Automata*. pp. 428–437. Springer, Cham (2018)
15. Marzolla, M.: Parallel Implementations of Cellular Automata for Traffic Models. In: Mauri, G., El Yacoubi, S., Dennunzio, A., Nishinari, K., Manzoni, L. (eds.) *Cellular Automata*. pp. 503–512. Springer, Cham (2018)
16. Morán, A., Frasser, C.F., Roca, M., Rosselló, J.L.: Energy-Efficient Pattern Recognition Hardware With Elementary Cellular Automata. *IEEE Transactions on Computers* **69**(3), 392–401 (2020)
17. Selvapeter, P.J., Hordijk, W.: Cellular automata for image noise filtering. In: 2009 World Congress on Nature & Biologically Inspired Computing (NaBIC). pp. 193–197 (2009)
18. Toffoli, T., Margolus, N.: *Cellular Automata Machines: A New Environment for Modeling*. The MIT Press, Cambridge, London (1987)
19. Vicari, A., Alexis, H., Del Negro, C., Coltelli, M., Marsella, M., Proietti, C.: Modeling of the 2001 lava flow at Etna volcano by a Cellular Automata approach. *Environmental Modelling & Software* **22**(10), 1465–1471 (2007)
20. Wolfram, S.: Cellular automata as models of complexity. *Nature* **311**, 419–424 (1984)
21. Wolfram, S.: Random sequence generation by cellular automata. *Advances in Applied Mathematics* **7**(2), 123–169 (1986)
22. Wolfram, S.: *A New Kind of Science*. Wolfram Media, Champaign (2002)